

Advanced Programming

CSE 201

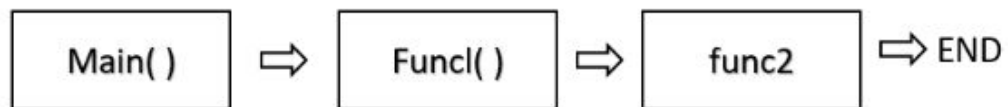
Lecture 16: Multithreading

Multithreading

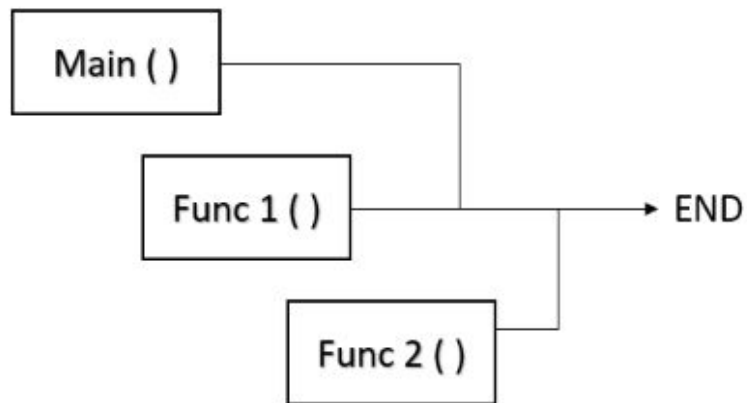
- Used to maximize the CPU utilization.
- We don't want our CPU to be in a free state; for example, Func1() is running and demands any input/output process. The CPU unnecessarily wait until Func1() completes its input/output operation.
- While Func1() completes its I/O operation, the CPU is free and not executing any thread. The efficiency of the CPU is decreased in the absence of multithreading.
- In the case of multithreading, if a thread demands any I/O operation, then the CPU will let the thread perform its I/O operation and will start the execution of a new thread. In this case, two threads are executing at the same time. If there is a waiting time in the new thread, the original thread can be reactivated.

Flow of control in java

- Without threading :



- With threading :



Creating a Threading

There are two ways to create a thread in java

1. By extending **Thread** class
2. By implementing **Runnable** interface

By Extending Thread Class

```
class MyThread1 extends Thread{
    @Override
    public void run(){
        int i =0;
        while(i<40000){
            System.out.println("111111111111");
            i++;
        }
    }
}

class MyThread2 extends Thread{
    @Override
    public void run(){
        int i =0;
        while(i<40000){
            System.out.println("2222222222222222");
            i++;
        }
    }
}

class thclass {
    public static void main(String[] args) {
        MyThread1 t1 = new MyThread1();
        MyThread2 t2 = new MyThread2();
        t1.start();
        t2.start();
    }
}
```

By Implementing Runnable Interface

```
class Runnable1 implements Runnable{
    public void run(){

        int i =0;
        while(i<40000){
            System.out.println("1111111111111111");
            i++;
        }

    }
}

class Runnable2 implements Runnable{
    public void run(){
        int i =0;
        while(i<40000){
            System.out.println("2222222222222222222222222222");
            i++;
        }

    }
}

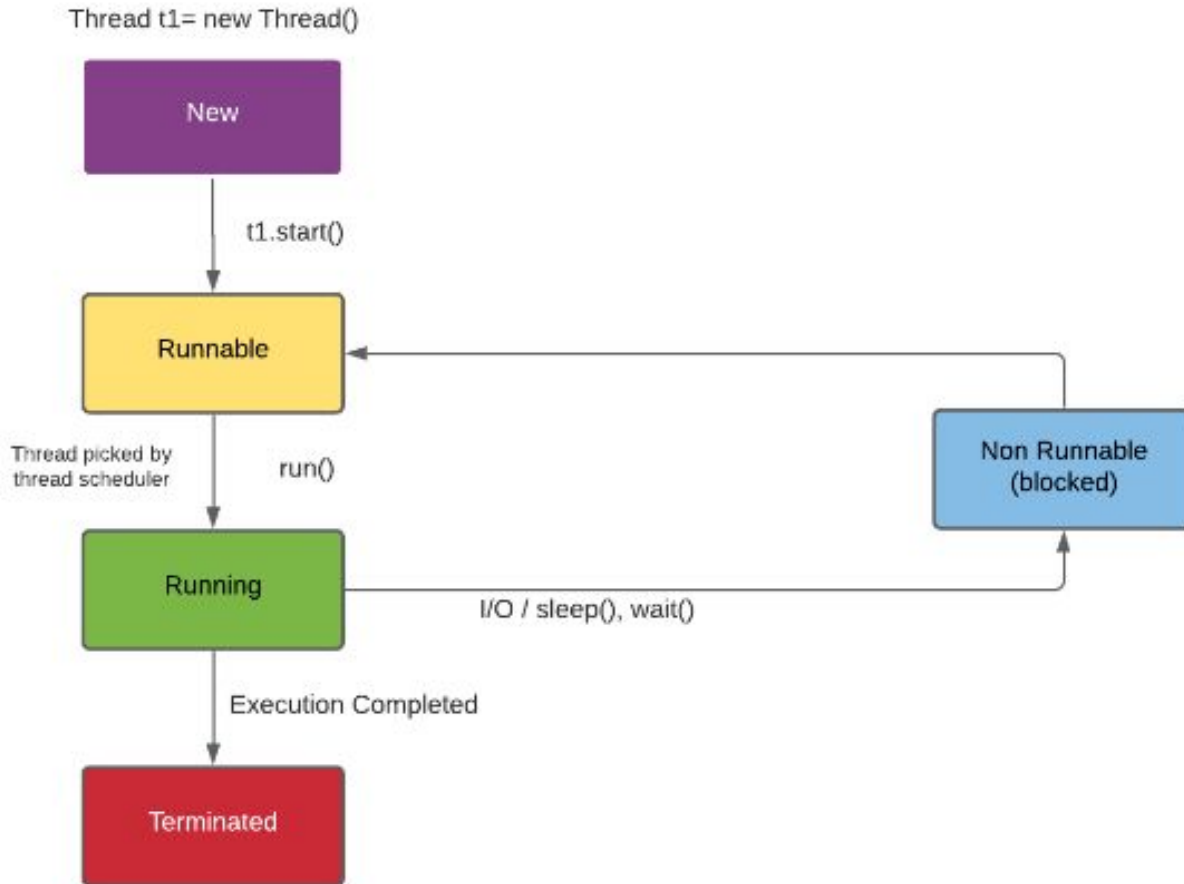
class rin {
    public static void main(String[] args) {
        Runnable1 r1 = new Runnable1();
        Thread t1 = new Thread(r1);

        Runnable2 r2 = new Runnable2();
        Thread t2 = new Thread(r2);

        t1.start();
        t2.start();

    }
}
```

Lifecycle of a thread



1. **New** - Instance of a thread is created, which is not yet started by invoking `start()` . In this state, the thread is also known as the born thread.
2. **Runnable** - After invocation of `start()` & before it is selected to be run by the scheduler.
3. **Running** - After the thread scheduler has selected it.
4. **Non-runnable** - thread alive, not eligible to run.
5. **Terminated** - `run()` method has exited.

Types of constructors

1. Thread ()
2. Thread (string)
3. Thread (Runnable r)
4. Thread (Runnable r, String name)

```
class MyThr extends Thread{
    public MyThr(String name){
        super(name);
    }
    public void run(){
        System.out.println("The name of the thread t is " + this.getName());
    }
}

class thcons {
    public static void main(String[] args) {
        MyThr t1 = new MyThr("Name1");
        MyThr t2 = new MyThr("Name2");
        t1.start();
        t2.start();
    }
}
```

In a Multithreading environment, all the threads which are ready and waiting to be executed are present in the Ready queue. The thread scheduler is responsible for assigning the processor to a thread. But the question is on what basis the thread scheduler decides that a particular thread will run before other threads. Here comes the concept of priority in the picture.

- ❖ Every single thread created in Java has some priority associated with it.
- ❖ The programmer can explicitly assign the priority to the thread. Otherwise, JVM automatically assigns priority while creating the thread.
- ❖ In Java, we can specify the priority of each thread relative to other threads. Those threads having higher priorities get greater access to the available resources than lower priorities threads.
- ❖ Thread scheduler will use priorities while allocating processor.

- ❖ The valid range of thread priorities is 1 to 10 (but not 0 to 10), where 1 is the least priority, and 10 is the higher priority.
- ❖ If there is more than one thread of the same priority in the queue, then the thread scheduler picks any one of them to execute.
- ❖ The following static final integer constants are defined in the Thread class:
 - MIN_PRIORITY: Minimum priority that a thread can have. Value is 1.
 - NORM_PRIORITY: This is the default priority automatically assigned by JVM to a thread if a programmer does not explicitly set the priority of that thread. Value is 5.
 - MAX_PRIORITY: Maximum priority that a thread can have. Value is 10.

You can set priority using any number between 1 and 10, not just these constants.

```
class MyThr1 extends Thread{
    public MyThr1(String name){
        super(name);
    }
    public void run(){

        while(true){

            System.out.println("Thank you: " + this.getName());

        }
    }
}

class thprio {
    public static void main(String[] args) {

        MyThr1 t1 = new MyThr1("H1");
        MyThr1 t2 = new MyThr1("H2");
        MyThr1 t3 = new MyThr1("H3");
        MyThr1 t4 = new MyThr1("H4");
        MyThr1 t5 = new MyThr1("H5 (most Important)");
        t5.setPriority(Thread.MAX_PRIORITY);
        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.MIN_PRIORITY);
        t3.setPriority(Thread.MIN_PRIORITY);
        t4.setPriority(Thread.MIN_PRIORITY);
        t5.setPriority(Thread.MIN_PRIORITY);

        t1.start();
        t2.start();
        t3.start();
        t4.start();
        t5.start();

    }
}
```

Introducing wait time though Thread.sleep().

T2 started right in the
first iteration in t1
because of the wait
time introduced.
Here, 1000 means
1000 ms, i.e. 1 sec.

th11
th22
th22
th22
th22
th22
th22
th22
th22
th22
th11
th11
th11
th11
th11
th11
th11
th11

```
class runnable1 implements Runnable{
    public void run(){
        int i=0;
        while (i<10){
            System.out.println(Thread.currentThread().getName());
            try{
                Thread.sleep(1000);
            }catch(Exception e){
                System.out.println(e);
            }
            i++;
        }
    }
}

class runnable2 implements Runnable{
    public void run(){
        int i=0;
        while (i<10){
            System.out.println(Thread.currentThread().getName());
            i++;
        }
    }
}

class Main{
    public static void main(String args[]) {
        runnable1 r1=new runnable1();
        runnable2 r2=new runnable2();

        Thread t1=new Thread(r1,"th11");
        Thread t2=new Thread(r2,"th22");

        t1.start();
        t2.start();
    }
}
```

join()

Forcing current thread to wait until a particular thread completes its job using join function.

In this case the main thread is the current thread and t1 is the particular thread.

Therefore, despite long waits in t1, t2 couldn't start, as main thread was waiting for t1 to finish before t2.start() statement can be reached.

```
class runnable1 implements Runnable{
    public void run(){
        int i=0;
        while (i<10){
            System.out.println(Thread.currentThread().getName());
            try{
                Thread.sleep(1000);
            }catch(Exception e){
                System.out.println(e);
            }
            i++;
        }
    }
}

class runnable2 implements Runnable{
    public void run(){
        int i=0;
        while (i<10){
            System.out.println(Thread.currentThread().getName());
            i++;
        }
    }
}

class Main{
    public static void main(String args[]) throws InterruptedException
    {
        runnable1 r1=new runnable1();
        runnable2 r2=new runnable2();

        Thread t1=new Thread(r1,"th11");
        Thread t2=new Thread(r2,"th22");

        t1.start();
        t1.join();
        t2.start();
    }
}
```

th11
th11
th11
th11
th11
th11
th11
th11
th11
th22
th22
th22
th22
th22
th22
th22
th22
th22